

Talks That Don't Fit Elsewhere

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Edition: 2024-07-03. Copyright © 2024 by Walter E. Brown. All rights reserved.

A little about me

- B.A. (math's); M.S., Ph.D. (computer science).
- Professional programmer for over 50 years, programming in C++ since 1982.
- Experienced in industry, academia, consulting, and research:
 - Founded a Computer Science Dept.; served as Professor and Dept. Head; taught and mentored at all levels.
 - Managed and mentored the programming staff for a reseller.
 - Lectured internationally as a software consultant and commercial trainer.
 - Retired from the Scientific Computing Division at Fermilab, specializing in C++ programming and in-house consulting.
- **Not dead — still doing training & consulting. (Email me!)**



Copyright © 2021 by Walter E. Brown. All rights reserved.

Emeritus participant in C++ standardization

- Written ~175 papers for WG21, proposing such now-standard C++ library features as `gcd/lcm`, `cbegin/cend`, `common type`, and `void_t`, as well as all of headers `<random>` and `<ratio>`.
- Influenced such core language features as *alias templates*, *contextual conversions*, and *variable templates*; recently worked on *requires-expressions*, `operator<=>`, and more!
- Conceived and served as Project Editor for *Int'l Standard on Mathematical Special Functions in C++* (ISO/IEC 29124), later incorporated into `<cmath>`.
- Be forewarned: Based on my training and experience, I hold some rather strong opinions about computer software and programming methodology — these opinions are not shared by all programmers, but they should be! 😊



Copyright © 2021 by Walter E. Brown. All rights reserved.



Exploring C++ Value Categories and the Expression Category Taxonomy

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Edition: 2024-06-24. Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

From low- to high-level programming

- Programming in machine language is replete with instructions that:
 - Read from/write to specific **memory addresses**, or ...
 - Read from/write to specific CPU **register addresses**.
- *I.e.*, an address is the **key** of a **key-value pair**.
- In higher-level languages, we programmers typically:
 - Use **symbols** (**names**, **identifiers**) in lieu of addresses, ...
 - Relying on their translator to choose addresses and ...
 - To map their program's symbols to those addresses.
- (In fact, assembly languages were commonly known as Symbolic Programming Systems 60+ years ago.)

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

8

Addresses are just bits, but so are data

- In any **von Neumann architecture** computer:
 - Bits denoting an address can't, by mere inspection, ...
 - Be distinguished from bits denoting data.
- By themselves, an address's bits may:
 - (a) Denote the **source** for a read/fetch instruction, or
 - (b) Denote the **target** for a write instruction, or ...
 - Be the bits that are read from or written to memory!
 - *N.B.*: The same bits may serve both purposes over time!
- Bits' **context** is **crucial** to their **correct interpretation**!
 - The bits' **type** is part of that context, of course, but ...
 - The **value category** is another part of that context.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

9

Type vs. value category

- The **type** of a bundle of bits tells us, e.g.,:
 - How many bits are involved and which are **salient**.
 - Whether and how those salient bits can be subdivided into **elements/members** (each with its own type).
 - Which algorithms will **query/observe** those salient bits, and which algorithms will **mutate** those salient bits.
- The **value category** of that bit bundle tells us, e.g.,:
 - Whether the bits denote an **address** or a **datum**.
 - More generally, whether those bits are suitable as **operands** for the **operator** to which they are bound.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

10

Early value category uses

- First introduced in **CPL** (Cambridge [later: **Combined**] Programming Language by Christopher Strachey, et al.):
 - A ~1963 ancestor of C by way of BCPL (Basic CPL) and B.
 - Used nomenclature **lvalue** and **rvalue** (non-lvalue in C) ...
 - ... to distinguish between expressions suitable for the left and the right sides, respectively, of an assignment.
 - In brief: it's an lvalue iff it has **identity** (e.g., a name).
- That means:
 - We can use the lvalue as a **key** ...
 - As part of a memory hardware **key-value pair** ...
 - To locate and read/write a corresponding **value** (rvalue).

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

11

Example

- Let's declare:
 - `int x = ...`
 `, y = ...;`
- Then consider these operands as subexpressions:
 - `x = 3`
 // ok: x is an lvalue (address), 3 is an rvalue (data)
 - `x = y`
 // ok: assignment needs an rvalue on the right,
 // so lvalue y must first convert (by reading memory)
 // ... to an rvalue like 3,
 // thus originating the "lvalue-to-rvalue conversion"

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

12

Analyzing some native C++98 operators (simplified)

- Each arithmetic, relational, and shift operator:
 - Takes two rvalues as operands.
 - Yields an rvalue result.
- Each assignment operator: *i.e., not const*
 - Takes one **modifiable lvalue** and one rvalue as operands.
 - Yields an lvalue result.
- Each increment and decrement operator:
 - Takes one modifiable lvalue operand.
 - Prefix forms `++k` and `--k` yield an lvalue result, but ...
 - Postfix forms `k++` and `k--` yield an rvalue result.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

13

Analyzing some more native C++98 operators

- The address-of operator `&`:
 - Takes one lvalue as operand.
 - Yields an rvalue result. (Often the same bits, though.)
 - Note: *not* considered an lvalue-to-rvalue conversion, because no memory is read; bits are simply reclassified.
- The indirection operator `*`:
 - Takes one rvalue as operand.
 - Yields an lvalue result. (Often the same bits, reclassified.)
- Operator `throw`:
 - Takes one rvalue as operand.
 - Yields an rvalue result (of type `void`).

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

14

Value categories lvalue/rvalue used to be enough

- ... until C++11 added `&&` (rvalue reference) types, ...
 - Plus the move semantics these types made possible.
- It became necessary to refine the value categories, ...
 - Yet without damaging long-used existing nomenclature.
 - "Key terms ... **lvalue** and **rvalue** retain their conventional meaning..." — Bjarne Stroustrup
- So C++11 introduced the **xvalue** category:
 - An **expiring value** (because it's about to be either replaced or destroyed) ...
 - And so its resources can be reused rather than await reclamation when the value is replaced or destroyed.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

16

Characteristics of an xvalue

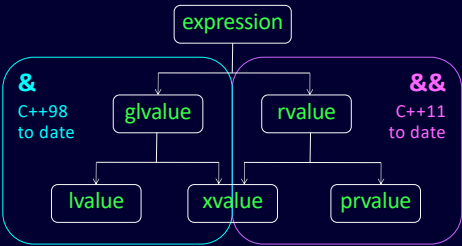
- An xvalue is just like an lvalue:
 - But we're allowed to treat it as if it were an rvalue.
- To manage that dichotomy, C++11 also introduced:
 - A **glvalue** (generalized lvalue), encompassing both the classical lvalue as well as the new xvalue categories.
 - A **prvalue** (pure rvalue), an rvalue that is not an xvalue.

The complete picture since C++11

(evaluated) expression	glvalue (generalized lvalue)	lvalue
	rvalue	xvalue (expiring value)
		prvalue (pure rvalue)

Note that the lvalue and rvalue categories are not at the same level; this asymmetry lets us preserve these terms' C++98 meanings.

C++ expression category taxonomy



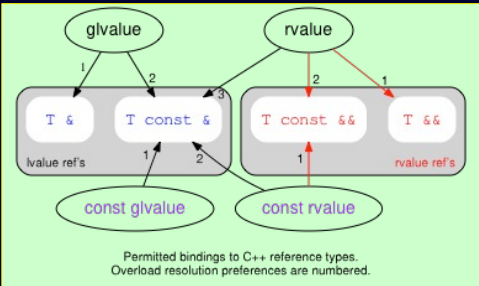
Reference bindings since C++98

Parameter type	If the argument expression yields a ...			
	mutable lvalue	const lvalue	mutable rvalue	const rvalue
T &	1 (pref)	X	X	X
T const &	2	✓	✓	✓

Reference bindings since C++11

Parameter type	If the argument expression yields a ...			
	mutable glvalue	const glvalue	mutable rvalue	const rvalue
T &	1 (pref)	X	X	X
T const &	2	✓	3	2
T &&	X	X	1	X
T const &&	X	X	2	1

Reference bindings, summarized





C++ Value Categories and the Expression Category Taxonomy

FIN

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Copyright © 2020-2024 by Walter E. Brown. All rights reserved.



An Adventure in Applied Detection

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Edition: 2024-06-24. Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

Background

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

A (very!) quick review of template specialization

- The compiler combines a **primary template**:
 - With (supplied or inferred) template arguments, ...
 - To **synthesize*** a **declaration** (not yet a definition!).
- Once a synthesized declaration is selected/requested:
 - The compiler looks for a matching programmer-supplied **specialization** (**definition**, either **explicit** or **partial**).
 - If one is found, the compiler **instantiates*** it.
 - Otherwise, the compiler **instantiates*** a specialization from the primary template's definition.
- *How? Each occurrence of a template parameter is replaced by its corresponding template argument.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

33

Another way to think about it

A cookie cutter :: a template.



Instantiation

Different dough? Different cookie!

Cookie dough :: template arg's. A cookie :: a specialization.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

34

The Detection Idiom

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

An example trait employing the detection idiom (C++11)

- `#include <cstddef>` // for `ptrdiff_t`
- `#include <type_traits>` // for `false_type` and `true_type`
- `template< class, class = std::ptrdiff_t >` // primary template
- `struct has_size : std::false_type { };`
- `template< class T >` // partial specialization
- `struct has_size< T, decltype(sizeof(T)) >`
- `: std::true_type { };`
- Recall that `sizeof(T)` is ill-formed for an object type `T` that is incomplete.
- So when `T` is incomplete, the partial spec. is ill-formed and (per *SFINAE*) is discarded/not further considered.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

36

Using that example trait based on the detection idiom

- `static_assert( has_size< int >::value );`
- `static_assert( not has_size< void >::value );`
- `static_assert( not has_size< int [ ] >::value );`
- `static_assert( not has_size< struct S >::value );`
- Please use such a `has_size` trait carefully!
 - For some types, this trait may give different results at different times during your program's compilation.
 - E.g., `false` after a class's forward declaration (incomplete) vs. `true` after that class's later definition (now complete).
 - Such inconsistency is problematic (violates ODR, etc.), so, for this example trait, inspect at most once per type.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

37

`void_t`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

`void_t` in context of the detection idiom [since C++17]

- `template< class ... >` // C++{11,14}, roll your own thusly using `void_t = void;`
- `template< class T, class = std::void_t<> >` // or just `void`
- `struct has_xyzzy`
- `: std::false_type { };`
- `template< class T >`
- `struct has_xyzzy< T, std::void_t< decltype(T::xyzzy) > >`
- `: std::true_type`
- `{ };`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

39

Oops!

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

The detection idiom used oddly [adapted from M. Caini]

- `struct S`
- `: has_xyzzy< S >` // note: CRTP! (accidental?)
- `{`
- `static constexpr int xyzzy = 42;`
- `};`
- This leads to the following paradoxical code:
 - // Apparently, `S` has no member named `xyzzy` ...
 - `static_assert( not S::value, "!" );`
 - // ... But really?
 - `static_assert( S::xyzzy == 42, "!" );` // passes!

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

41

void_t in context of the detection idiom, redux

- `template< class T, class = std::void_t<> >`
`struct has_xyzzy`
`: std::false_type`
`{`
`static_assert( has_size<T>::value // true iff T is complete`
`, "has_xyzzy needs a complete object type"`
`" to give a meaningful result!" );`
`};`
- `template< class T >`
`struct has_xyzzy< T, std::void_t< decltype(T::xyzzy) > >`
`: std::true_type { };`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

42

The detection idiom used oddly, redux

- `struct S`
`: has_xyzzy< S > // type S is still incomplete here!`
`{`
`static constexpr int xyzzy = 42;`
`};`
- *// Apparently, S has no member named xyzzy ...*
`static_assert( not S::value, "!" ); // no longer compiles!`
- *has_xyzzy needs a complete object type to give a meaningful result!*
- *// ... Really?*
`static_assert( S::xyzzy == 42, "!" );`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

43

C++20

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

As of C++20, detection is a language feature

- Its syntax is termed a *requires-expression*:
 - A *requires-expression* is an ordinary expression (technically a *prvalue expression*) of type `bool`.
- ⚠ (Don't confuse with a *requires-clause*, which was designed mainly to apply concepts to algorithms, etc.)
- `requires { ... }` is one form of *requires-expression*:
 - The braces enclose one or more requirements to be assessed.
 - The *requires-expression* yields `true` iff each requirement is well-formed and then found to be satisfied.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

45

Detection à la C++20

- `template< class T >`
`struct has_xyzzy`
`: std::bool_constant< requires`
`{ typename T::xyzzy; }`
`>`
`{`
`static_assert( has_size<T>::value // true iff T is complete`
`, "has_xyzzy needs a complete object type"`
`" to give a meaningful result!" );`
`};`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

46

Slightly more idiomatically

- `template< class T >`
`concept complete_obj_type = requires { sizeof(T); };`
- `template< class T >`
`struct has_xyzzy`
`: std::bool_constant< requires`
`{ typename T::xyzzy; }`
`>`
`{`
`static_assert( complete_obj_type<T>`
`, "has_xyzzy needs a complete object type"`
`" to give a meaningful result!" );`
`};`

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

47

Finally: could instead make a concept

- `template< class T >`
`concept has_xyzzy = requires { typename T :: xyzzy; } ;`
- But this won't produce a custom diagnostic if it's inadvertently applied to an incomplete type:
 - In some applications, such a diagnostic is desirable, ...
 - But in others, a diagnostic may be unwanted.
- So it's still useful to know several implementations of the **detection idiom**.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

48



An Adventure in Applied Detection

FIN

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

Tales of Subscribing

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Edition: 2024-06-24. Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

~1972

*Should array indices start at 0 or 1?
My compromise of 0.5 was rejected
without, I thought, proper consideration.*

— STAN KELLY-BOOTLE
(NÉ STANLEY BOOTLE)

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

Circa 1972 (from memory: changed since?)

- I had come from a Fortran background:
 - When that language was still spelled **FORTRAN II**.
 - It had by 1972 evolved into **FORTRAN IV** (1962).
 - (Yes, there had been a **FORTRAN III** [1958?], but AFAIK that version was never released outside IBM.)
- Fortran arrays were defined via a **DIMENSION** stmt:
 - The array's **extent** was specified explicitly, but its **lower bound** was **implicitly always 1** (i.e., 1-origin), so ...
 - A Fortran array's extent is also its **upper bound**.
 - E.g., **DIMENSION A(5)**

A(1)	A(2)	A(3)	A(4)	A(5)
------	------	------	------	------

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

54

Then I learned PL/I

- My first time seeing (a) the `/* ... */` comment syntax, and (b) semicolons as **statement terminators**.
- Names of PL/I variables, including arrays, were introduced via a **DECLARE** statement:
 - Also abbreviated **DCL** (because the obvious **DEC** was sadly taken as the abbreviation for **DECIMAL** ☹).
- A PL/I array could be **declared with a lower bound as well as an upper bound**:
 - The extent was then calculated $e \equiv ub - lb + 1$.
 - E.g., **DCL A(15,20) BIN FIXED(31); /* 31 bits + sign */**

A(15)	A(16)	A(17)	A(18)	A(19)	A(20)
-------	-------	-------	-------	-------	-------

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

55

Soon after, I learned K&R C...

- C arrays are declared with what it terms the “bound”:
 - But that “bound” is not the upper bound; it’s really the intended extent. (BTW, the C++ std library uses the proper term!)
- As we know, C arrays are implicitly 0-origin:
 - Since $e = b$, the upper bound is calculated $ub \equiv e - 1$.
- C introduced me to the notion of contextual decay:
 - Namely, that the name of an array can convert (decay) to a pointer to the array’s leading (not 1st) element.
 - So subscripting is elegantly defined as $A[k] \equiv *(A + k)$.
 - By +’s commutativity, $A[k] \equiv *(A + k) \equiv *(k + A) \equiv k[A]$.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

56

I recoded several Fortran pgms into C

- Adjusting 1-origin to 0-origin was especially painful:
 - I had no time budget to recode the logic afresh.
 - Specifically, each Fortran subscript expression $A(k)$ had to become $A[k-1]$.
 - And there were quite a lot of them to adjust!
- Although a largely mechanical transformation, our tools were then inadequate to automate the process:
 - E.g., we needed to avoid adjusting Fortran function call expressions, because ...
 - Those calls, too, used parentheses as an operator.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

57

What did I do?

- Rather than adjusting each subscript by -1:
 - For each Fortran declaration of the form
`DIMENSION A(N)`
I provided a corresponding C declaration of the form
`double A0[N], * const A1 = A0 - 1;`
 - (C didn’t yet have `const`, but you get the idea.)
- Now I could use the same subscript to:
 - Do C’s 0-origin subscripting via the `A0` name, and ...
 - Verify against Fortran’s 1-origin subscripting via the `A1` name, since ...
 - The names’ suffixes conveyed which origin was used.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

58

This technique solved our problems brilliantly at the time

- It relieved me, and every future code reader, of the need to check each subscript for the correct offset.
- But this technique won’t work in today’s C++:
 - `double A0[N], * const A1 = A0 - 1; // ill-formed!`
 - We can’t form the initializing expression `A0-1`.
 - Why? Because today’s “abstract machine is rather picky, as enforced by `constexpr`, ...
 - “although back then compilers wouldn’t have cared and would have generated correct code anyways.”

— Stephan T. Lavavej

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

59

Tales of Subscripting

FIN

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Copyright © 2020-2024 by Walter E. Brown. All rights reserved.



Neither East nor West: I Recommend Post-Positive `const`

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Edition: 2024-06-24. Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

Two overlapping opinions

♪ East is east and west is west
And the wrong one I have chose ... ♪

— Jay Livingston and Ray Evans,
Buttons and Bows (from *Paleface*), 1947

Oh, East is East, and West is West,
and never the twain shall meet, ...;
But there is neither East nor West, ...

— Rudyard Kipling,
The Ballad of East and West, 1889

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

70

Very brief grammar review

- An **adjective's** role is to describe (**modify**) a noun:
 - E.g., heavy book, green tea, tall building
- Most (but not all) adjectives have 3 forms:
 - E.g., heavy book, heavier book, heaviest book
 - E.g., unique book, but neither uniquer nor uniquest book (because “unique” means “one of a kind”)

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

71

These 3 forms have names

- It's the **superlative** when 3+ items are involved:
 - E.g., heaviest book (of these 3 or more books)
 - E.g., tallest building (of these 3 or more buildings)
- It's the **comparative** when 2 items are involved:
 - E.g., heavier book (of these 2 books)
 - E.g., taller building (of these 2 buildings)
- It's the **positive** form when there's exactly 1 item:
 - E.g., heavy book, green tea, tall building

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

72

Where to place an adjective?

- An adjective generally precedes its noun:
 - E.g., English, German, Russian, Afrikaans, Cherokee, ...
- An adjective generally follows its noun:
 - E.g., Latin, French, Spanish, Hebrew, Arabic, Swahili, ...
- But language rules are full of exceptions:
 - An English adjective following its noun is a post-positive.
 - (If there's an intervening verb, it's a predicate adjective.)

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

73

In some languages, the position affects the meaning!

- E.g., in Italian:
 - Post-positive: un bambino piccolo → a young child
 - Pre-positive: un piccolo bambino → a small child
- E.g., in French:
 - Post-positive: une voiture ancienne → an antique car
 - Pre-positive: une ancienne voiture → a former car
- E.g., in C++:
 - ✓ Post-positive: char * const // modifies the pointer
 - ✓ Post-positive: char const * // modifies the pointee
 - ✓ Pre-positive: const char * // modifies the pointee
 - ✗ Pre-positive: const (char *) // modifies the pointer

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

74

Aliases don't help

- “A **typedef-name** is ... a synonym for another type”:
 - E.g., using char_ptr = char *; // since C++11
 - Equivalently, typedef char * char_ptr; // since C
- So these are the same:
 - const char_ptr // immutable pointer
 - char_ptr const // immutable pointer
- But after alias substitution they are not the same:
 - const char * // immutable pointee
 - char * const // immutable pointer

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

75

The post-positive is not at all uncommon

- Culinary arts: fettuccine Alfredo, whisky sour, cherries jubilee, eggs Benedict, steak tartare, veal piccata, something {bland, spicy, sweet, salty, ...} ...
- Religion: God Almighty, devil incarnate, angels aplenty, ...
- Organizations: Knights Templar, Amnesty International, {Alcoholics, Gamblers, Overeaters} Anonymous, ...
- Law: court martial, fee simple, proof positive, notary public, malice aforethought, {persons, parts} unknown, payment due, heir apparent, letters patent, ...
- Heraldry: lion {rampant, passant, statant}, griffin argent, bend {dexter, sinister}, ...

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

76

Something useful: nothing strange: everyone present

- Positions: professor emeritus, minister plenipotentiary, poet laureate, {attorney, postmaster, surgeon, secretary} general, queen {regnant, consort, regent}, president elect, ...
- Appellations: Henry VIII; Martin Luther King, Junior; Brothers Grimm; Alexander the Great; Johnny come-lately; ...
- Phrases: arms akimbo, buildings ablaze, roses galore, time immemorial, times past, words unspoken, pound sterling, accounts {payable, receivable}, body politic, arms wide open, battle royal, visitors sad to leave, code {red, blue}, ...
- Titles of works: Brideshead Revisited, Enemy Mine, Paradise Lost, Regained, Rabbit Redux, Prometheus Unbound, ...

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

77

And in poetry ...

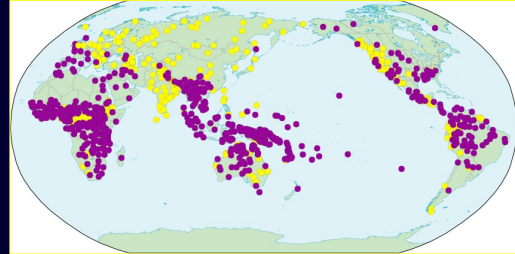
- “Something old, something new,
something borrowed, something blue”
- “This is the forest primeval.
The murmuring pines ... in garments green, ...
Stand ... with voices sad and prophetic,
Stand like harpers hoar, ...”
— *Evangeline: A Tale of Acadie*
Henry Wadsworth Longfellow

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

78

A survey of 1000+ languages

[D. Ross]



- Yellow ½: 340 languages with adjective-noun order.
- Purple ¾: 768 languages with noun-adjective order.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

79

In my own code

- I discovered, long ago, that most C++ declarations are best read mostly right-to-left:
 - using ptr_to_void = void *;
 - using arr_of_10_ptr_to_dbl = double * [10];
 - using fctn_taking_ref_to_int_returningflt = float (int&);
- At the same time, I respect precedence:
 - using ptr_to_arr_of_10_dbls = double (*) [10];
 - using ref_to_fctn_taking_ref_to_dbl_returningchar = char (&) (double &);

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

80

So I long ago concluded ...

- That it's best to use the **post-positive**:
 - using immutable_ptr_to_char = char * const;
 - using ptr_to_immutable_char = char const *;
- In my own code, I've consistently used only the post-positive for many decades now, ...
- So, for all the reasons mentioned, I recommend using **post-positive const** as a best practice.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

81



Neither East Nor West: I Recommend Post-Positive const

FIN

Walter E. Brown, Ph.D.
< webrown.cpp @ gmail.com >



Copyright © 2020-2024 by Walter E. Brown. All rights reserved.



It's Not Only Unique: Tips to Improve Technical Writing

Walter E. Brown, Ph.D.
< webrown.cpp @ gmail.com >



Edition: 2024-06-24. Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

We programmers write more than just code

- Our technical writing takes many forms nowadays:
 - Program comments
 - Reference documentation
 - Blogs
 - Tutorial documentation
 - Proposals
 - Bug reports/issue tracking
 - Publications
 - Correspondence
 - Progress reports
 - Wording a C++ standard!
- Although our technical knowledge is required, ...
- Ultimately, it's all about communication, right?
- So we might as well be good at it! Here are some tips.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved. 86

Tip #1: spell correctly

- Why?
 - Incorrect spelling diverts readers' attention and thus interferes with our communication with them.
 - Unwanted reaction: "What is the intent of that word? Is that a typo, or have I misunderstood something?"
 - Really unwanted: "Is this really worth reading?"
- The most commonly misspelled English word:
 - Has only three letters: its (or should that be it's?).
 - Rule: it's always means it is or it has — no exceptions!
 - In contrast, its always means belonging to it!
- Also: TL;DR is correctly spelled Abstract or Summary!

Copyright © 2020-2024 by Walter E. Brown. All rights reserved. 87

Tip #2: use affirmative (positive) phrasing

- Why?
 - Because inverted logic is often confusing to a reader.
 - "It's not the case that our project is over budget."
 - Say what you mean: "Our project is under budget."
- Be extra careful using the word "not":
 - "I don't think that"
 - I stop reading when you tell me that you "don't think"!
 - Instead, tell me what you do think/believe/opine/...!
 - Avoid "I don't want"; tell me what you do want!
 - Tell me how to fix my mistake, not only that I made one!

Copyright © 2020-2024 by Walter E. Brown. All rights reserved. 88

Tip #3: arrange phrases to reflect your intent

- Why?
 - To avoid miscommunication and embarrassments.
 - "For sale: antique chair
by elderly lady with cane bottom."
 - Better: "For sale by elderly lady: antique chair with...."
- More examples from publications:
 - "Nearly 600,000 men in the United States get vasectomies each year." [Seems unnecessary, no?]
 - "A declaration of a block variable with linkage that has an initializer is ill-formed." [dcl.init.general]/5

Copyright © 2020-2024 by Walter E. Brown. All rights reserved. 89

Examples from a single publication

- “Scientists have found that studying high-resolution images of **whales from space** is a feasible way to estimate their populations.” [Published 2020-07-31.]
- “A combination of satellite images and deep learning computer algorithms can spot **elephants from space** as well as the human eye.” [Published 2021-01-24.]
 - And what’s a human eye doing in space to be spotted?
- (I’ve since stopped my subscription, lest I find more examples of **animals and body parts from space**.)

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

90

Be careful placing even single words!

[JE. Brennecke]

- A 7-word sentence: “I hit him in the eye yesterday.”
- Let’s insert another word, trying each of the 8 places:
 - **Only I** hit him in the eye yesterday. [No one else hit him.]
 - I **only hit** him in the eye yesterday. [I did nothing else.]
 - I hit **only him** in the eye yesterday. [I hit no one else.]
 - I hit him **only in** the eye yesterday. [I hit nowhere else.]
 - ⋮
 - I hit him in the **only eye** yesterday. [Is he a Cyclops?]
 - ⋮

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

91

A final favorite example

- An ad for an outdoors gadget for the yard:
 - “Prevent damage ... from burrowing rodents with [this] electronic stake that emits vibration and sound that’s intensely annoying to underground **rodents up to 100 feet in diameter**.”
 - “Requires 4 ‘D’ batteries, not included.”
- Reader comment:
 - “I suppose for rodents of greater than 100 feet in diameter you need the nuclear powered version.”

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

92

Tip #4: use each word correctly

- Why?
 - To promote reader comprehension of your intent, and ..
 - To avoid possible reader confusion as to your intent.
- Egregious example:
 - **Unique** means *one-of-a-kind*. (Note the *uni-* prefix.)
 - Therefore, if it’s unique, there’s no other like it.
 - It’s a binary quality: there are no degrees of uniqueness.
 - So let’s avoid such meaningless nonsense as **very unique** and **fairly unique** and **somewhat unique** and so on.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

93

A closing thought

“Choosing the right word,
and the right word order, ...
could make an enormous difference
in conveying an image or an idea.”



— Hon. Ruth Bader Ginsburg

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

95

So please remember that “It’s Not Only Unique”

- It’s: spelling is important.
- Not: affirmative phrasing is important.
- Only: word order is important.
- Unique: word meaning is important.

Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

96



It's Not Only Unique:
Tips to Improve Technical Writing

FIN

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Copyright © 2020-2024 by Walter E. Brown. All rights reserved.

Talks That Don't Fit Elsewhere

FIN

Walter E. Brown, Ph.D.

< webrown.cpp @ gmail.com >



Copyright © 2024 by Walter E. Brown. All rights reserved.